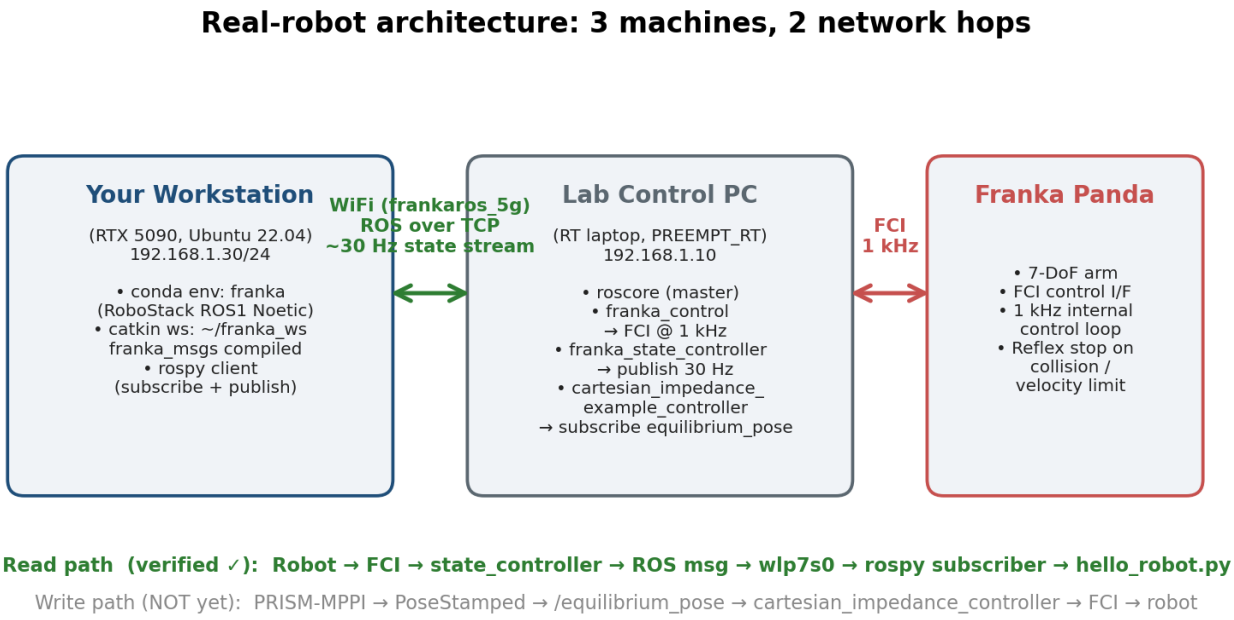
PRISM-JEPA Real-Robot ROS Communication

Status briefing • 2026-05-06 • Author: Yuhai • Sister project: *prism-jepa* (LeWM substrate)

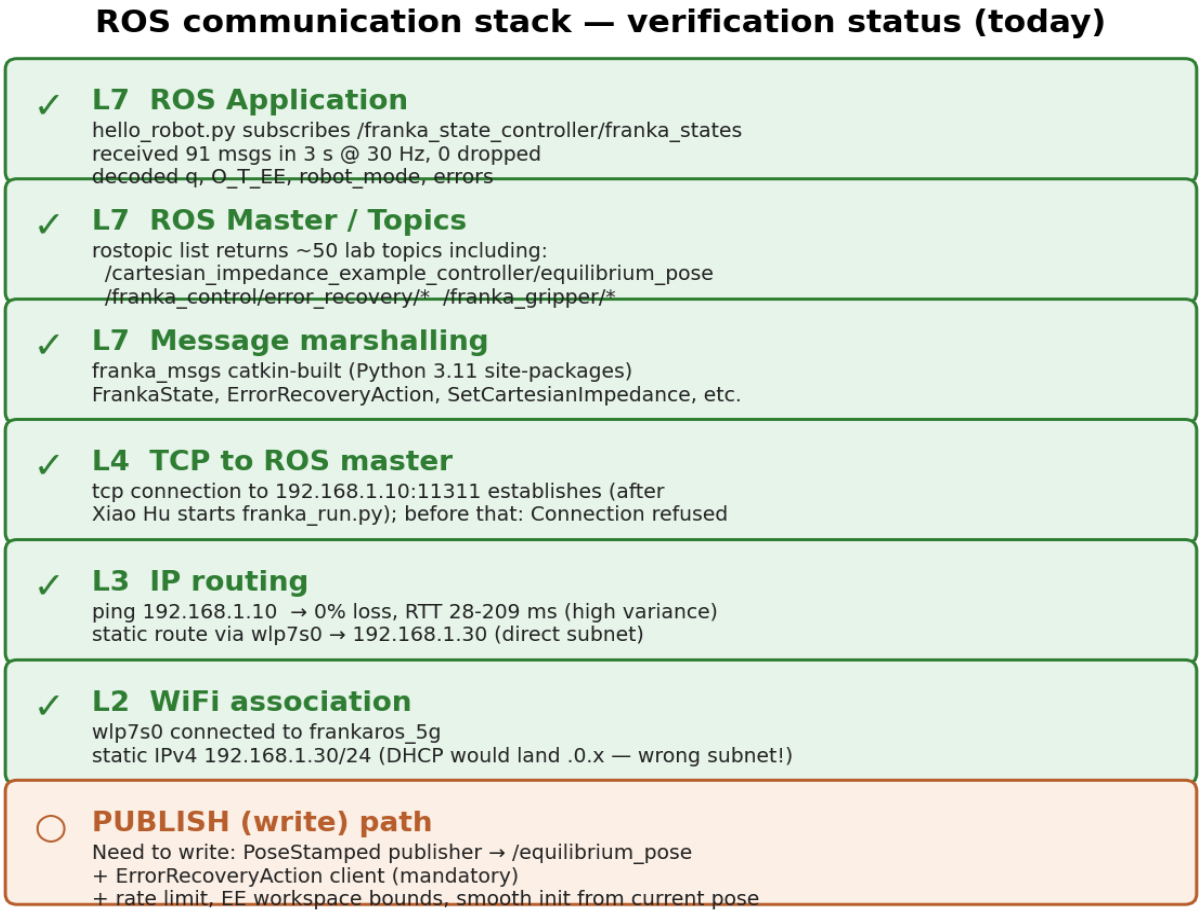
One-line status. Today the full read-only ROS communication path from this workstation to the lab Franka was verified: WiFi → IP routing → ROS master → topic subscribe → 30 Hz real robot state. The publish (write) path is intentionally not yet implemented — it requires Xiao Hu's *franka_run.py* as a reference for lab convention before we can safely send commands.

1. Architecture: three machines, two network hops



Our workstation is a **ROS client**. Real-time torque control is the lab control laptop's job (PREEMPT_RT-patched kernel + libfranka over FCI). We publish high-level EE pose targets at ~10 Hz over WiFi; the lab laptop tracks them at 1 kHz internally and publishes 30 Hz state back. We never need RT scheduling on this side — that's the whole point of separating concerns.

2. ROS communication stack — verification status



All read-side layers are verified end-to-end with real Franka data. The single remaining row is the publish path — by design, since writing a publisher means sending commands the robot will execute, and we need Xiao Hu's reference script first to align with lab convention (frame, rate, smoothing).

3. Network setup — what we configured

Step	What	Value	Constraint
1	Connect to lab WiFi	frankaros_5g	must be physically in the lab
2	WiFi IPv4 (manual, NOT DHCP)	192.168.1.30/24	must equal $ROS_HOSTNAME
3	ROS env vars	MASTER=192.168.1.10:11311 HOSTNAME=192.168.1.30	set by ~/franka_ws/setup_franka.sh
4	L3 reachability	ping 192.168.1.10	0% loss expected
5	L7 reachability	rostopic list	needs lab roscore running

The classic gotcha we hit today: WiFi DHCP gave us 192.168.0.102 (different subnet from the control PC at 192.168.1.10), so ping failed even though the WiFi was connected. The fix is the manual static-IP step (item 2 above) — explicitly required by the lab email but easy to miss.

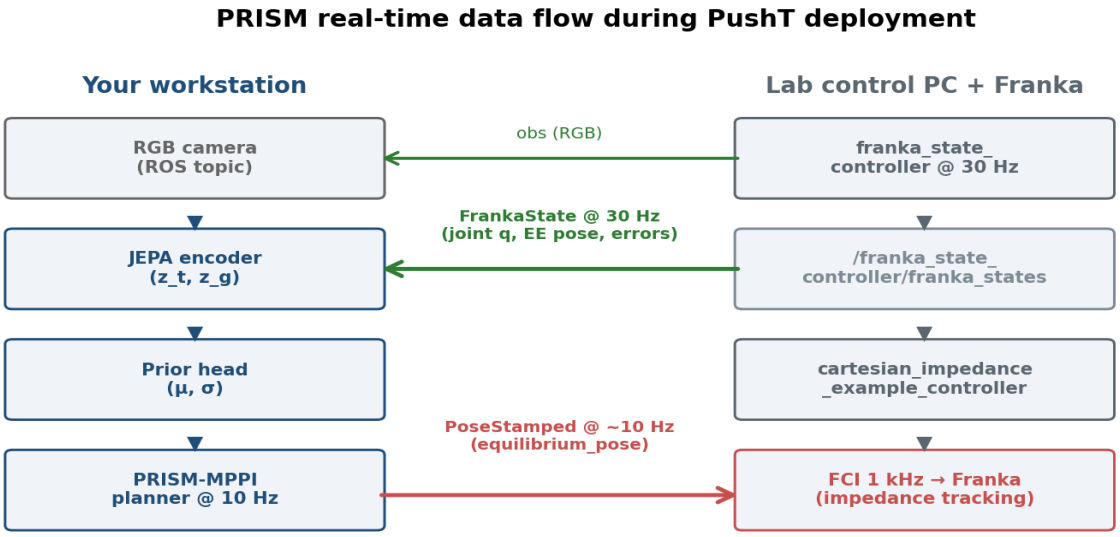
4. What's running on the lab control PC (after Xiao Hu starts the stack)

Component	Status today	What it does	We use it as
roscore	✓ on 192.168.1.10:11311	ROS message bus / master	the address we set $ROS_MASTER_URI to
franka_control	✓ running	talks to robot via FCI at 1 kHz	we don't touch it directly
franka_state_controller	✓ publishing 30 Hz	publishes joint q, EE pose, errors	subscribe /franka_state_controller/franka_states
cartesian_impedance_example_controller	✓ loaded	tracks an EE pose target via impedance	publish PoseStamped to .../equilibrium_pose
error_recovery action server	✓ on /franka_control/error_recovery/*	clears reflex / collision latch	MUST call after any reflex; not optional
franka_gripper	✓ on /franka_gripper/*	open/close/grasp action interface	PushT doesn't need this

5. Real-robot state we read today

Field	Value	Interpretation
q (7 joints, rad)	[-0.002, -0.785, 0.000, -2.364, -0.003, +1.572, +0.777]	Standard Franka home pose [0, -π/4, 0, -3π/4, 0, π/2, π/4]
EE position (m)	(+0.306, -0.001, +0.483)	30 cm in front of base, 48 cm above table
robot_mode	2 (MOVE)	Motor power on, ready to receive commands
control_command_success_rate	1.000	Controller is happy
current_errors	False	No latched errors
State publish rate measured	~30 Hz over 3 s, 91 msgs	Matches franka_state_controller default; 0 dropped
WiFi RTT	min 28 ms / max 209 ms / mdev 81 ms	High variance — flag for closed-loop testing later

6. Operational data flow during PushT deployment

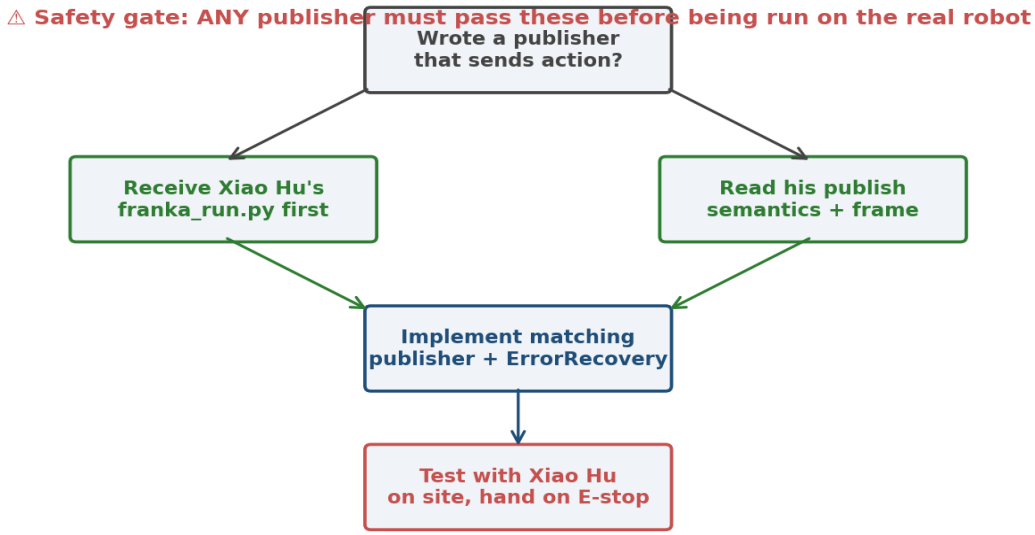


Read leg (verified): camera RGB → JEPA encode → (z_t, z_g) → prior head → (μ, σ) → PRISM-MPPI planner. State subscription confirmed at 30 Hz today. **Write leg (pending):** planner output → PoseStamped @ ~10 Hz → /cartesian_impedance_example_controller/equilibrium_pose → tracked at 1 kHz on the lab side → robot moves.

7. What's intentionally not yet done

Pending item	Why we didn't do it today
Publisher to /equilibrium_pose	Sending a command moves the robot. We need Xiao Hu's franka_run.py first to align frames, rate, and
ErrorRecoveryAction client	Mandatory companion to any publisher: Franka latches errors on collision and refuses new commands
Read Xiao Hu's franka_run.py	He hasn't shared the file yet (his keyboard-control reference). We don't guess at lab convention.
Camera ROS topic	Need to ask Xiao Hu whether the lab has an RGB camera already publishing, or if we set one up. PRIS
Real-robot training data	Sim-trained JEPA + world model + prior head will not transfer. Plan: 200 real teleop demos → re-train s

8. Safety gating before any publisher runs



This is the gate every PRISM publisher passes through before being run against the real Franka. The ErrorRecovery client must be implemented first; the EE workspace bounds must be coded in; the first PoseStamped sent must be from the robot's current EE pose to avoid an instantaneous tracking jump; and Xiao Hu (or another lab member with E-stop access) should be present for the first 5–10 closed-loop trials.

9. Anticipated questions and prepared answers

Q	A
What's verified end-to-end today?	L2 (WiFi) → L3 (IP) → L4 (TCP to ROS master) → L7 (subscribe FrankaState). Real robot at home pose, M
Why no publisher yet?	Intentional. Need Xiao Hu's franka_run.py for lab convention; need to co-implement ErrorRecoveryAction clie
Is WiFi reliable enough?	30 Hz state with 0 packet loss today, but RTT variance is high (28-209 ms). Flagged as risk for closed-loop p
Why not run RT control on our workstation?	GPU inference (JEPA + MPPI) preempts CPU and would trigger Franka safety stops. Plus we'd need a PREE
What's the timeline to first Push-T closed-loop?	Receive Xiao Hu's script (1-2 days) + align publish semantics (½ day) + minimal publisher with error_recover
Sim-to-real gap?	The sim-trained JEPA encoder will not generalize to real Franka pixels. Need ~200 real teleop demos and re

Files referenced in this brief: ~/franka_ws/setup_franka.sh, ~/franka_ws/hello_robot.py, ~/franka_ws/mock_franka_publisher.py, ~/franka_ws/src/franka_ros/.